



Concurrency: An Introduction

Week 10: Threads, Race Conditions & Critical Sections

lectura.id/course/os

Program Studi Teknik Informatika
Institut Teknologi Sumatera

2026

OUTLINE

Peta materi pertemuan ini

Capaian Pembelajaran

Tujuan setelah pertemuan selesai

1. Menjelaskan apa itu thread dan bagaimana thread berbeda dari proses.
2. Mengidentifikasi sumber daya apa yang di-*share* dan apa yang bersifat private pada thread.
3. Membaca kode C sederhana dengan thread dan memprediksi kemungkinan outputnya.
4. Mendefinisikan *race condition* dan memberikan contoh konkretnya.
5. Menjelaskan mengapa `counter++` bisa menghasilkan hasil salah ketika dijalankan oleh dua thread.
6. Mendefinisikan *critical section* beserta tiga syarat solusinya.

Fokus Kelas Ini

Kita fokus pada **perbedaan thread dan proses**, **race condition**, dan **critical section** secara step-by-step.

Pembuka & Motivasi

Mengapa Kita Perlu Concurrency?

Dunia nyata tidak sekuensial

Aplikasi nyata yang perlu concurrency:

- **Browser:** Download file + render halaman + handle input user → **bersamaan**
- **Server web:** Ratusan request masuk secara bersamaan
- **Video game:** Render grafis + update AI + handle input → paralel
- **Smartphone:** Background service + UI responsiveness

Bayangkan...

Jika browser tidak bisa download sambil merender, maka:

- Download selesai → baru render
- User freeze saat download
- Aplikasi terasa "beku"

Sangat jelek untuk UX!

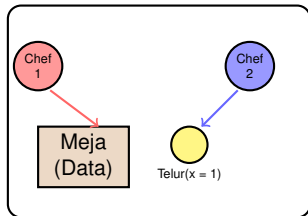
Tantangan

Concurrency bawa masalah baru: jika dua thread mengakses data yang sama, hasil bisa tidak terduga.

Analogi: Dua Chef di Dapur yang Sama

Mengapa concurrency itu tricky

Dapur (Shared Memory)



Skenario Masalah:

- **Chef 1** ambil telur terakhir
- **Chef 2** juga ambil telur (waktu bersamaan)
- Hasil: telur hilang, data rusak

Analoginya:

- Chef = **thread**
- Dapur = **shared memory**

Agenda Kelas Hari Ini

Apa yang kita akan pelajari dalam 100 menit

Bagian 1: Thread Abstraction (25 mnt)

Apa itu thread? Bagaimana thread berbeda dari proses? Apa yang *shared* dan apa yang *private*?

Bagian 2: Race Condition (30 mnt)

Apa itu *race condition*? Kenapa `counter++` bisa salah? Mari kita lihat di level instruksi.

Bagian 3: Critical Section (20 mnt)

Apa itu *critical section*? Apa tiga syarat solusinya?

Bagian 4: Ringkasan & Latihan (15 mnt)

Review konsep, latihan soal, dan Q&A.

Checkpoint 0

Cek pemahaman sebelum lanjut

1. Apa perbedaan utama antara **concurrency** dan **parallelism**?
2. Mengapa browser perlu concurrency?
3. Dalam analogi dapur, apa representasi dari thread dan shared memory?

Kunci Jawaban Singkat

1. Concurrency = switching antar task cepat (illusion of parallelism); Parallelism = truly simultaneous.
2. Agar download, render, dan input handling bisa terjadi tanpa user freeze.
3. Chef = thread; Dapur + ingredients = shared memory.

Thread Abstraction

Thread vs Process

Perbedaan dan keuntungan thread

Masalah dengan Process murni:

- **Fork** membuat **copy penuh** address space → **mahal**
- Komunikasi antar proses (IPC) kompleks: pipe, socket, shared memory
- Context switch antar proses **berat**: harus ganti page table

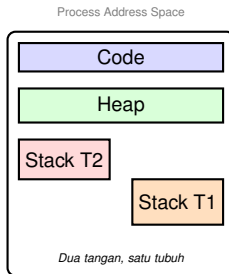
Thread sebagai solusi:

- Thread = unit eksekusi **di dalam** proses
- Berbagi address space yang sama
- Lebih ringan: buat dan switch lebih cepat

Aspek	Process	Thread
Address space	Masing-masing	Shared
Buat over-head	Besar	Kecil
Context switch	Berat	Ringan
Komunikasi IPC	kompleks	Shared memory
Crash satu	Isolasi	Crash proses

Shared vs Private dalam Thread

Apa yang di-share, apa yang private?



Di-shared antar thread (1 proses)

- Code segment
- Heap
- Global variables
- File descriptors

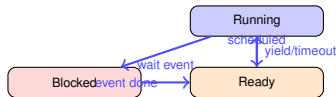
- Signal handlers

Thread Control Block & Thread States

Bagaimana OS mengelola thread?

Thread Control Block (TCB):

- Analogus dengan PCB untuk process
- Berisi: register state, stack pointer, program counter, thread ID, state



Thread States:

- **Running** — sedang dijalankan oleh CPU
- **Ready** — siap jalan, tapi CPU lagi jalan thread lain
- **Blocked** — menunggu event (I/O, lock, dll)

Context Switch antar Thread:

1. Simpan register thread lama ke TCB

2. Thread Abstraction

3. Muat register thread baru dari TCB

Pthread API: Membuat dan Join Thread

Contoh sederhana dengan pthread

```
#include <stdio.h>
#include <pthread.h>

void *my_thread(void *arg) {
    char *name = (char *) arg;
    printf("Thread %s berjalan\n", name);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, my_thread, "A");
    pthread_create(&t2, NULL, my_thread, "B");
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Main selesai\n");
    return 0;
}
```

Penjelasan Kode

- `pthread_create()`: buat thread baru
- `pthread_join()`: tunggu thread selesai

Pertanyaan Penting!

Apakah output **selalu**: Thread A
Thread B Main selesai ?

*Jawab: **Tidak!*** Urutannya bisa berubah. Ini disebut **non-determinism**.

Checkpoint 1

Cek pemahaman Bagian 1

1. Sebutkan tiga sumber daya yang di-**shared** antar thread dalam satu proses.
2. Sebutkan dua sumber daya yang **private** per thread.
3. Pada contoh kode `pthread_create`, apakah urutan output Thread A dan Thread B selalu sama? Mengapa?
4. Bagaimana OS melakukan context switch antar thread?

Kunci Jawaban Singkat

1. Shared: code, heap, file descriptors
2. Private: stack, register, PC
3. Tidak — bergantung pada scheduler. Inilah *non-determinism*.
4. Simpan TCB lama, load TCB baru, ubah stack pointer.